

Курсов проект по предмета „Операционни системи“ на Мартин Михалков

Вариант: „7 задачи“

Задача 1: hexdump

Използвани библиотеки:

```
#include <fcntl.h> // File control, defines file manipulation and file opening control, and the access modes
#include <unistd.h> // Unix standard, defines all sys calls
#include <string.h>
#include <stdio.h>
```

Функция, която принтира съобщение на екрана, като автоматично намира дължината на подадения string.

```
// A function which writes a string to a given file descriptor by automatically finding the string's length.
void write_str(int fd, const char *str) {
    // write() requires that an address is given for it to read.
    write(fd, str, strlen(str)); // since str is already a pointer, we don't need to use '&'
}
```

Използвани променливи:

- fd – файлов дескриптор на отворения файл.
- bytes_read – брой байтове от 0 до BUFFER_SIZE, които остават за текущия буфер (текущия ред).
- buf[] - стринг, съдържащ текущия буфер.
- hex_chars – използван за преобразуването, в шестнадесетичен формат.

```
int fd;
ssize_t bytes_read; // Will contain the number of bytes that have been read with read
()
unsigned char buf[BUFFER_SIZE];
const char hex_chars[] = "0123456789ABCDEF"; // Used for the byte -> hex conversion
```

```

if (argc != 2) {
    write_str(2, "hexdump: Missing file argument\n"); // Writes the string to STDERR
    return 1;
}

fd = open(argv[1], O_RDONLY); // Opens the file using the
if (fd < 0) {
    perror("hexdump"); // Prints the error that occurred prefixed with "hexdump".
    return 1;
}

```

Проверява дали точно един параметър е наличен (първият елемент в argc е пътя до изпълнимия файл)
Отваря файла само за четене.

```

while ((bytes_read = read(fd, buf, BUFFER_SIZE)) > 0) {
    // Loops while there is at least one byte left unread from the file

```

Цикъла се повтаря докато има поне един байт, останал за четене.

```

/* 1. Prints the hex representation. It cycles for the whole BUFFER_SIZE */
for (i = 0; i < BUFFER_SIZE; i++) {
    if (i < bytes_read) {
        // If there are bytes which are not yet converted to hex, it converts them
        char hex[3];
        hex[0] = hex_chars[buf[i] >> 4];
        hex[1] = hex_chars[buf[i] & 0x0F];
        hex[2] = '\0';
        write_str(1, hex);
    } else {
        /* Else it fills missing bytes with spaces (2 spaces per byte) */
        write_str(1, "  ");
    }
}

```

За всеки цикъл на while, for цикъла се изпълнява за всички битове в буфера. bytes_read запазва броя на байтовете в текущия буфер. bytes_read е или колкото BUFFER_SIZE, или по-малко. Ако в текущия буфер все още има байтове за печатана, те се превръщат в шестнадесетичен формат в if оператора. Накрай принтираме преобразувания байт на стандартния изход (файлов дескриптор 1).

Ако наличният брой байтове е по-малък от буфера, заместваемте всеки байт с „ „.

```

/* Handle spacing between bytes */
if (i < BUFFER_SIZE - 1) { // We use "-1" so that no space is put after the last byte
    // After the byte hex representation is printed, the spacing after it is decided
    if (i == 7) {
        write_str(1, "  "); /* 2 spaces between 8th and 9th */
    } else {
        write_str(1, " "); /* 1 space otherwise */
    }
}
}

```

След принтирането на шестнадесетичния формат на даден байт, определяме, дали да принтираме „ “ или „“, в зависимост дали текущо принтираният байт е осмия (с индекс 7) или не, съответно.

```

/* 3 spaces to separate hex and ASCII */
write_str(1, "   ");

```

Отделя шестнадесетичното представяне с ASCII представянето на битовите принтирайки „ „.

```

/* 2. ASCII representation loop */
for (i = 0; i < BUFFER_SIZE; i++) { // Write all ASCII chars that will fit in BUFFER_SIZE
    if (i < bytes_read) {
        // If there are still bytes left in the current BUFFER_SIZE, they should be written
        if (buf[i] >= 32 && buf[i] <= 127) {
            // If the ASCII chars have a code between 32 and 127 (visible symbols), then print them.
            write(1, &buf[i], 1); // Here we are giving the address of a char in the char array.
        } else {
            // Else substitute them with "."
            write_str(1, ".");
        }
    } else {
        // If there aren't characters anymore to print, pad the line with "."
        write_str(1, ".");
    }
}
}

```

оая

За всеки байт от текущия буфер или принтира ASCII формата на байта, или принтира „.“, ако няма повече байтове в текущия буфер. За байтовете, които трябва да бъдат принтирани, С проверява дали те могат да бъдат представени като символи (ако имат код между 32 и 127) или не, и ако не, програмата замества байта с „.“.

```

write_str(1, "\n"); // At the end of each line a line feed is written.

```

Накрая на всяка итерация на while(), се принтира нов ред.

```
close(fd); // Closing the file.
```

Затваря се отворения файл.

Задача 2: reverse

Използвани
библиотеки.

```
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>
```

```
if (argc != 2) {
    write(2, "reverse: Invalid arguments\n", 27);
    return 1;
}

int fd = open(argv[1], O_RDWR);
if (fd < 0) {
    perror("reverse");
    return 1;
}
```

Проверява се дали е подаден точно един параметър. Отваря файла подаден като аргумент в режим на четене и писане и връща грешка ако не може да се оправи.

```
off_t file_size = lseek(fd, 0, SEEK_END);
if (file_size < 0) {
    perror("reverse");
    close(fd);
    return 1;
}
```

Намираме размера на файла, чрез използване на `lseek`, която връща позицията на курсора след преместването. `Lseek` в случай премества курсора на текущия файл дескриптор в края на файла.

```
off_t left = 0; // the first byte index
off_t right = file_size - 1; // the last byte index
```

Създавам left и right, запазващи индекса на първия и последния блок.

```
// Loop until the two pinter don't meet each other in the middle
while (left < right) {
```

Цикъла прави итерации докато индекса на left и индекса на right не се срещнат.

За всяка итерация на цикъла създаваме две

```
// contains the left and right char to be swapped
char left_char, right_char;

// Moves the pointer to the left byte and reads it
lseek(fd, left, SEEK_SET);
read(fd, &left_char, 1);

// Moves the pointer to the right byte and reads it
lseek(fd, right, SEEK_SET);
read(fd, &right_char, 1);

// Swap and write them back to opposite locations
lseek(fd, left, SEEK_SET);
write(fd, &right_char, 1);

lseek(fd, right, SEEK_SET);
write(fd, &left_char, 1);

// Move markers inward
left++;
right--;
```

променливи, които съдържат байта определен от индексите на left и right съответно. Първият lseek премества курсора до позицията на left и запазва байта в left_char.

Вторият lseek премества курсора до позицията на right и запазва байта в right_char.

Третият lseek премества отново курсора на позицията на left, но този път презаписва байта с байта записан в right_char.

Четвъртият lseek премества курсора на позицията на right и презаписва байта с байта записан в left_char.

Накрая left и right се преместват навътре.

```
close(fd); // close the file
```

Затваряне на файла.

Задача 3: prun

Използвани
библиотеки.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>
```

```
if (argc < 2) return 0;
```

Проверява дали имо поне един подаден параметър.

```
int num_cmds = argc - 1;
pid_t *pids = malloc(num_cmds * sizeof(pid_t));
```

Запазва броя на подадените параметри в num_cmds и заделя динамично памет за PID-тата на всички процеси, които ще бъдат създадени (по един процес за всеки подаден параметър).

```
// Launch all commands in parallel
for (int i = 0; i < num_cmds; i++) {
```

Цикъла се изпълнява за всеки процес.

Fork() връща PID-то на дъщерния процес който е създал. Ако fork() върне число < 0, значи има грешка при създаването на процеса.

```
pids[i] = fork();

if (pids[i] < 0) {
    perror("prun: fork failed");
    continue;
}
```

```

if (pids[i] == 0) {
    // 1. Mute command output to pass tests
    int null_fd = open("/dev/null", O_WRONLY);
    if (null_fd >= 0) {
        dup2(null_fd, STDOUT_FILENO);
        close(null_fd);
    }

    // 2. Let the shell parse and execute the string automatically
    char *args[] = {"/bin/sh", "-c", argv[i + 1], NULL};
    // replaces the current child process image with the shell program.
    // creates the shell with args[0] and then passes the command string to it
    execvp(args[0], args);

    perror("prun: execvp failed");
    exit(1);
}

```

Кода в if оператора се изпълнява само на дъщерния процес. Бащерният го пропуска.

Отваря се файла „/dev/null” към който пренасочваме стандартния изход, така, че изхода от подадените команди да не се принтира. След това затваряме „/dev/null”.

След това съставяме args, който представлява команда, изпълняваща подадената команда в /bin/sh.

I ехесвр презаписваме програмата на дъщерния процес, с командата, която искаме да изпълним. Подадени са два параметъра на ехесвр: първо подаваме командата извъкваща „/bin/sh“ и после на нея подаваме останалите параметри.

```
int remaining = num_cmds;
```

Създаваме променлива, която следи процесите, които все още не са завършили. Отначало се приема, че никои процес не е завършил.

```
while (remaining > 0) {
```

Цикъла се повтаря докато има все още изпълняващи се процеси.

```

int status;
// waitpid cleans child processes, so that they don't become zombies
// -1, means waitpid waits for any child process, not a specific one
// $status is the address to which to store the child's exit code
// 0 means to use the default behaviour, i.e. freeze until a child finishes
// If the child process did not exit normally (because it crashed), this macro tells you which signal killed
it (e.g., Segmentation Fault, Killer signal, etc.).
pid_t finished_pid = waitpid(-1, &status, 0);

if (finished_pid < 0) break;

```

Създава променлива status, която съдържа изходната информация от waitpid, който изчиства дъщерните процеси, така, че да не станат „зомби“ процеси (процеси, които са приключили, но все още са в таблицата с процеси). „-1“, означава, че функцията ще работи за всеки процес и записва изходния код в status. „0“, означава, че се използват настройките по подразбиране. Waitpid връща или -1 (грешка), или 0 (процеса не си е сменил състоянието; при променяне на настройките), или >0 (PID-то на дъщерния процес, чието състояние се е променило.)

Проверява се дали има грешка при изпълнението.

```

// Find which argument matches this finished PID
for (int i = 0; i < num_cmds; i++) {
    if (pids[i] == finished_pid) {
        // WIFEXITED (What if exited), returns non-zero if the child
        // exited normally and 0 if it crashes or was forcefully
        // killed by a signal.

        // WEXITSTATUS (Wait, exit status), extracts the actual exit code (0 for success, 1 or 2 for errors)

        // WTERMSIG (Wait, terminated by signal), If the child process did not exit normally (because it crashed), this macro
        tells you which signal killed it (e.g., Segmentation Fault, Killer signal, etc.).

        // if the process was killed successfully, exit_code contains the exit code, else it contains the process that killed it.
        int exit_code = WIFEXITED(status) ? WEXITSTATUS(status) : WTERMSIG(status);
        printf("[%d] \"%s\" exited with status %d\n", finished_pid, argv[i + 1], exit_code);
        pids[i] = -1; // Mark as processed
        remaining--;
        break;
    }
}

```

Цикъл, който намира кой индекс от масива pids е равен на PID-то на завършения процес.

exit_code съдържа или изходния код, ако процеса е завършил успешно, или сигнала, който го е терминиран, ако процеса не е завършил успешно.

Принтира се PID-то, командата и изходния код на процеса.

Задава PID-то на текущия процес да е равно на „-1“ в pids.

Намаляваме remaining с 1. Използваме break; за да прекъснем цикъла, защото вече сме намерили процеса, който търсим.


```
free(pids); // Free the allocated space
```

Освобождава се алокираното място.

Задача 4: pipe2

Използвани библиотеки.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h> // wait() and waitpid(), WIFEXITED, WEXITSTATUS, WTERMSIG
#include <sys/types.h> // pid_t and others
```

```
// Check that exactly two arguments are passed
if (argc != 3) {
    write(2, "pipe2: Invalid arguments\n", 25);
    return 1;
}

// getting the two arguments
char *cmd1 = argv[1];
char *cmd2 = argv[2];
```

Проверява дали са подадени точно два параметъра и създава два масива, които съдържат двата параметъра.

```

int pipefd[2];
// pipe() creates an anonymous pipeline in RAM and gives two new file descriptors.
// you read pipefd[0]'s fd and write to pipefd[1]
// Pipes will block if empty: If a process tries to read() from a pipe but the
// writer hasn't sent anything yet, the reader will freeze (block) and sleep until data
// arrives.

// Closing is mandatory: If the writer process finishes but forgets to close its
// copy of pipefd[1], the reader process will hang forever waiting for more data, never
// realizing that the writer is done.

if (pipe(pipefd) < 0) {
    perror("pipe2: pipe failed");
    return 1;
}

```

Създава се масив от тип `int`, който съдържа файловете дескриптори на двата края на анонимна тръба. Проверява дали тръбата е създадена успешно.

```

// Fork the first child (Left Command: cmd1)
pid_t pid1 = fork();
if (pid1 < 0) {
    perror("pipe2: fork 1 failed");
    return 1;
}

```

Клонира текущия процес и създава нов дъщерен процес. Проверява дали е създаден успешно.

```

if (pid1 == 0) {
    // Inside Child 1: Redirect stdout to the write end of the pipe
    // duplicates write end to the standard OUTPUT!!!!
    // It says: close this child's connection to the terminal screen, and redirect its standard
    // output into the write-end of our pipe instead.
    dup2(pipefd[1], STDOUT_FILENO);

    // Close the two pipe ends
    close(pipefd[0]);
    close(pipefd[1]);

    // Execute using shell execution to automatically handle string spaces
    // NULL marks the absolute end of the array. execvp relies on this NULL marker to know when
    // to stop reading arguments.
    char *args[] = {"/bin/sh", "-c", cmd1, NULL};
    // replace the current running program with a brand-new program.
    execvp(args[0], args);

    perror("pipe2: execvp 1 failed");
    exit(1);
}

```

Кода в този if оператор се изпълнява само от първия дъщерен процес. Клонира се стандартния изход в записващия край на тръбата. След това тръбата се затваря за първия процес. **ТОВА, ЧЕ Е ЗАТВОРЕНА ТРЪБАТА ЗА ПЪРВИЯ ПРОЦЕС, НЕ ЗНАЧИ, ЧЕ Е ЗАТВОРЕНА ЗА ВТОРИЯ ПРОЦЕС.**

След това се създава масив, който съдържа командата, която ще презапише процеса. С `execvp` стартираме „/bin/sh“ и му подаваме следващите аргументи. Ако има грешка с промяната на програмата в процеса, `C` ще стигне до `perror` и ще изведе грешка и ще върне код „1“.

```

// Fork the second child (Right Command: cmd2)
pid_t pid2 = fork();
if (pid2 < 0) {
    perror("pipe2: fork 2 failed");
    return 1;
}

```

Създаваме втори дъщерен процес, като клонираме бащиния. Проверяваме дали процеса е успешно създаден.

```

if (pid2 == 0) {
    // Inside Child 2: Redirect stdin to the read end of the pipe
    // Closing pipefd[0] afterwards just cleans up the temporary variable; the connection to the
    // pipe remains open via STDIN_FILENO.
    // Redirects the read end to the standard INPUT!!!!
    // Redirect standard INPUT so it reads from the pipe.
    dup2(pipefd[0], STDIN_FILENO);

    // Before Child two closes the pipe ends it duplicates the read end onto standard input.
    // closing the two pipe ends
    close(pipefd[0]);
    close(pipefd[1]);

    // Execute using shell execution
    char *args[] = {"/bin/sh", "-c", cmd2, NULL};
    execvp(args[0], args);

    perror("pipe2: execvp 2 failed");
    exit(1);
}

```

Кода в този if оператор се изпълнява само от втория дъщерен процес. Насочваме стандартния вход на процеса към края за четене на тръбата. След това затваряме двата края на тръбата за този процес.

След това се създава масив, който съдържа командата, която ще презапише процеса. С `execvp` стартираме `„/bin/sh“` и му подаваме следващите аргументи. Ако има грешка с промяната на програмата в процеса, `C` ще стигне до `perror` и ще изведе грешка и ще върне код „1“.

```

// Closing the two pipe ends
close(pipefd[0]);
close(pipefd[1]);

```

Извън if операторите затваряме двата края на тръбата, която така и не е използвана от бащиния процес.

```

int status1, status2;

// Explicitly wait for both children by their specific PIDs
waitpid(pid1, &status1, 0);
waitpid(pid2, &status2, 0);

// Calculate exit code based on the rightmost command (cmd2)
int exit_code = 0;
if (WIFEXITED(status2)) {
    exit_code = WEXITSTATUS(status2);
} else if (WIFSIGNALED(status2)) {
    // we use 128 + signal, so the shell can distinguish signal termination from ordinary exit
    // values.
    exit_code = 128 + WTERMSIG(status2);
}

return exit_code;

```

Създаваме status1 и status2, които съдържат изходното състояние на двата процеса.

След това проверяваме дали втория процес е излязал със статус код „0” и ако да, връщаме изходния код на втория процес.

В противен случай проверяваме дали сигнал е прекъснал процеса и ако да задаваме exit_code да е равно на 128 + сигнала.

Накрая връщаме exit_code.

Задача 5: worker

Използвани библиотеки.

```

#include <stdio.h>
#include <unistd.h>
#include <signal.h>

```

```
// volatile tells the compiler not to optimize the variable
// if modern compilers see that a var. is not used withing a loop, they store
// it in really fast register instead of checking it from RAM
// Global flags strictly using the required type
// Else the compiler will replace while(var), with while(1) if it sees var never changes.

//sig_atomic_t tells C, that the var, can be read, or written in a single "atomic" operation, a single uninterrupted state.
volatile sig_atomic_t keep_running = 1;
// It flips to 1 the moment SIGUSR1 hits, signaling the main loop to print a status update.
volatile sig_atomic_t usr1_received = 0;

// Signal handler for SIGINT and SIGTERM
void handle_term() {
    keep_running = 0;
}

// Signal handler for SIGUSR1
void handle_usr1() {
    usr1_received = 1;
}
```

Създавам променлява `keep_running`, която ще казва на програмата дали да продължава да се изпълнява. Създавам променлива `usr1_received`, която сигнализира дали сигнала `usr1` е бил получен.

`Volatile` означава, че компилатора няма да се опитва да оптимизира кода, като заменя някои променливи в цикли с `while(1)` ако променливите не се сменят.

`sig_atomic_t` означава че тези променливи ще бъдат записани или променени на един път, а не на няколко операции.

`handle_term()` и `handle_usr1()` са handler-и, които имат за цел да променят стойността на гореспоменатите променливи при подаване на някакъв сигнал.

```
// Line buffer stdout so text prints immediately even when run in background
setvbuf(stdout, NULL, _IOLBF, 0);

// Map the handlers to a specific action
struct sigaction sa_term;
sa_term.sa_handler = handle_term;
// initializes a sigset_t so it contains no signals. there is also
sigfillset, sigaddset and sigdelset to modify the set.
sigemptyset(&sa_term.sa_mask);
sa_term.sa_flags = 0;
```

Със `setvbuf` изхода от `printf` ще се изпраща към стандартния изход при срещане на нов ред, вместо да чака буфера да се напълни.

Създава се структура `sigaction`, която описва какво да се случи при получаване на даден сигнал.

Със `sigemptyset` се инициализира маската от сигнали като празна, т.е. по време на изпълнението на обработчика няма да бъдат блокирани допълнителни сигнали.

Със `sa_term.sa_flags = 0` не се задават специални флагове и се използва стандартното поведение на `sigaction()`.

```
// Register both shutdown signals
// installs signal handlers
// sa_handler: the function runs when the signal arrives
sigaction(SIGINT, &sa_term, NULL);
sigaction(SIGTERM, &sa_term, NULL);

struct sigaction sa_usr1;
sa_usr1.sa_handler = handle_usr1;
// sa_mask: sigset_t of signals to block while the handler runs
sigemptyset(&sa_usr1.sa_mask);
// behavior modifiers ()
sa_usr1.sa_flags = 0;

// Register the status signal
sigaction(SIGUSR1, &sa_usr1, NULL);
```

Със `sigaction` указваме кой handler да се изпълни при подаване на определен сигнал.

Създава се нова конфигурация за сигнал `sa_usr1` и му се настройва handler.

Със „`sa_usr1.sa_flags = 0`“, се указва, че не се използват специални опции, а се прилага стандартното поведение на `sigaction()`.

```
int counter = 0;
```

Създава се променлива брояч.

```
while (keep_running) {
```

Създава се цикъл, който се изпълнява докато `keep_running` е 1, тоест докато handler-а не е извикан.

```

// sleep(1) c #include <stdio.h>
// We only ti #include <stdlib.h>           en't checked
yet.          #include <string.h>
sleep(1);     #include <unistd.h>           hing else
// Check if w #include <time.h>
if (!keep_rur #include <sys/socket.h>
    break;    #include <netinet/in.h>
}

counter++;
printf("tick %d\n", counter);

// Check if SIGUSR1 was caught during this second
if (usr1_received) {
    printf("status: tick %d\n", counter);
    usr1_received = 0; // Reset flag
}

```

Вътре в цикъла: изчаква се 1 секунда, проверява се дали `keep_running` все още е 1 (това е нужно защото докато програмата е неактивна, заради `sleep(1)`, и сигнал я събуди, ако няма проверка за `keep_running`, С ще увеличи брояча с 1 преди да спре програмата), брояча се увеличава с 1 и се принтира неговата стойност. Накрая се проверява дали е получен потребителския сигнал (ако е получен `usr1_received` ще е равен на 1 от handler-a), и съобщението се принтира на екрана, както и `usr1_received` става равно на 0.

```

// Graceful shutdown message using the last valid counter value
printf("shutdown after %d ticks\n", counter);

```

При излизане от цикъла, тоест при получаване на сигнал за прекъсване, се изписва колко секунди са минали от началото до терминирането на програмата.

Задача 6: `timerserver` и `timeclient`

timeserver.c:

Използвани
библиотеки.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <time.h>
#include <sys/socket.h>
#include <netinet/in.h>
```

```
if (argc != 2) {
    fprintf(stderr, "timeserver: missing port\n");
    return 1;
}
```

Проверява се дали е подаден точно един аргумент.

```
int server_fd = socket(AF_INET, SOCK_STREAM, 0);
if (server_fd < 0) {
    perror("timeserver");
    return 1;
}
```

Създава се сокет, от AF_INET семейството, което указва на сокета, че ще използва протоколния стек на Ipv4. Указваме, че искаме транспортният протокол да е TCP със SOCK_STREAM. Понеже единственият протокол в SOCK_STREAM е TCP, избираме първия (използвайки 0). Накрая проверяваме дали дали сокета е създаден успешно.

```

int opt = 1;

// When a TCP connection closes, the operating system kernel doesn't
// release the network port instantly. It forces the port into a protective
// state called TIME_WAIT for 1 to 2 minutes.

// This is a built-in safety feature of TCP to catch any late or delayed
// data packets still floating around the internet from the old connection,
// preventing them from accidentally corrupting a brand-new process.

// This line tells the server to immediately release the port,
// when the server process closes.
setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

```

По подразбиране когато TCP връзката се затвори, използвания порт е недостъпен за нови връзки за около 1-2 минути, което означава, че при незабавно повторно изпълнение на програмата, тя ще даде грешка, казвайки, че порта вече се използва. Със `setsockopt` указваме, че искаме порта да бъде достъпен за използване, незабавно след затваряне на сокета. Използва се променливата `opt`, тъй като `setsockopt` изисква адрес от който да прочете дадена стойност за трети параметър.

```

// Creates addr structure for IPv4 data. Wipes any garbage data in RAM.
struct sockaddr_in addr = {0};
addr.sin_family = AF_INET;
// Which NIC the server should listen on.
addr.sin_addr.s_addr = INADDR_ANY; // 0.0.0.0 (listen from all NICs)
// Big endian (stores the MSB (not bit) first)
// Little endian (stores the LSB (not bit) first)
// Most protocols use BE.
// Most CPUs use LE.
// htons flips the bytes.
// atoi() gets the port number and converts it to int.
addr.sin_port = htons(atoi(argv[1]));

```

След създаване на сокета, трябва да му зададем адрес на който да слуша и порт. Това правим със структурата `sockaddr_in`. Първо я инициализираме, премахвайки случайни данни записани на нейното място в RAM паметта. След това задаваме семейство и IP адресите на локалните мрежови карти на устройството, на които искаме сървъра да слуша. `INADDR_ANY` е

заместител за стойността „0.0.0.0“, която конфигурира сокета да слуша на всички налични мрежови карти.

След това се задава порт. Тъй като повечето CPU-та запазват информация във формат Little Endian, което означава, че байта с най-малка тежест се записва пръв, а повечето мрежови протоколи използват формат Big Endian, което означава, че байта с най-голяма тежест се записва пръв, трябва да обърнем реда на байтовете, които репрезентират порта, на който да се слуша, подаден като параметър.

Със функцията `atoi`, се превръща номера на порт, запазен като `string`, в `int`.

```
// binds a socket which is only created in a namespace, with an IP address.
if (bind(server_fd, (struct sockaddr *)&addr, sizeof(addr)) < 0) {
    perror("timeserver");
    close(server_fd);
    return 1;
}
```

Със функцията `bind`, програмата свързва сокета с горесъздадената структура и проверява дали операцията е минала успешно.

```
// 1 means that at most 1 connection to the socket can be initiated at a
time
// assigns the socket to be listening
listen(server_fd, 1);
```

Указва се на сокета да слуша за връзки, като най-много една връзка може да бъде иницирана наведнъж през него.

```
// Creates a socket based on the first request to the server socket.
int client_fd = accept(server_fd, NULL, NULL);
if (client_fd < 0) {
    perror("timeserver");
    close(server_fd);
    return 1;
}
```

Създава се сокет, базиран на първата заявка за връзка към сървърния сокет и се проверява дали създаването е било успешно.

```
// Get and format time
time_t t = time(NULL);
struct tm *tm_info = localtime(&t);
char buf[64];
strftime(buf, sizeof(buf), "%Y-%m-%d %H:%M:%S\n", tm_info);
```

Първият ред взима текущото време от системния часовник във формат „Unix Epoch Time“, който представлява броят секунди, изминали от 1 януари 1970 година 0:00 часа.

Със вторият ред, се създава структура, която използва датата запазена във формат „Unix Epoch Time“ като по този начин, може да се извлече информация за текущата дата във четим за човека формат.

С третия ред се създава един стринг, който ще бъде изпратен по клиентския сокет до клиента.

Функцията strftime взима като входни параметри, буфера, в който искаме да запазим изхода, формата, в който искаме да представим текущата дата в буфера и структурата, от която да се взима информация за датата.

```
write(client_fd, buf, strlen(buf));

close(client_fd);
close(server_fd);
```

Буфера се изпраща по клиентския сокет и накрая и двата сокета се затварят.

timeclient.c:

Използвани
библиотеки.

```
42 #include <stdio.h>
41 #include <stdlib.h>
40 #include <string.h>
39 #include <unistd.h>
38 #include <sys/socket.h>
37 #include <netinet/in.h>
36 #include <netdb.h>
```

```
if (argc != 3) {  
    fprintf(stderr, "timeclient: missing arguments\n");  
    return 1;  
}
```

Проверява дали са подадени точно два параметъра.

```
// Makes DNS query  
struct hostent *server = gethostbyname(argv[1]);  
if (!server) {  
    fprintf(stderr, "timeclient: host not found\n");  
    return 1;  
}
```

Ако е подадено име на хост, вместо IP адрес, проверява дали съществува запис в локалната DNS таблица и ако не бъде намерено нищо там, прави lookup през мрежата. Ако на `gethostbyname()` се подаде IP адрес, функцията просто ще го запази без да прави DNS заявка. Ако има грешка при създаването на структурата, програмата връща грешка на `STDERR`.

```
// creates a socket  
int sock_fd = socket(AF_INET, SOCK_STREAM, 0);  
if (sock_fd < 0) {  
    perror("timeclient");  
    return 1;  
}
```

Създава се сокет, от `AF_INET` семейството, което указва на сокета, че ще използва протоколния стек на `Ipv4`. Указваме, че искаме транспортният протокол да е `TCP` със `SOCK_STREAM`. Понеже единствения протокол в `SOCK_STREAM` е `TCP`, избираме първия (използвайки `0`). Накрая проверяваме дали сокета е създаден успешно.

```
// creates a struct
struct sockaddr_in addr = {0};
addr.sin_family = AF_INET;
memcpy(&addr.sin_addr.s_addr, server->h_addr_list[0], server->h_length);
addr.sin_port = htons(atoi(argv[2]));
```

Създава структура, която съдържа IP адреса и порта, на който ще изпраща клиентския сокет. Първо структурата се инициализира. После информацията от DNS lookup-а се копира в структурата. Накрая се задава порта, като се използва htons за да информацията, запазена във формат „Little Endian” да бъде преобразувана във формат „Big Endian”. Със функцията atoi, порта подаден като параметър, се превръща от string в int.

```
char buf[128];
ssize_t n;
while ((n = read(sock_fd, buf, sizeof(buf) - 1)) > 0) {
    buf[n] = '\0';
    printf("%s", buf);
}
```

Създава се буфер, в който да бъде запазен отговора на сървъра. Цикълът прочита изпратените данни блок по блок. Когато сървърът приключи, n става равно на 0, което прекъсва цикъла. Вътре в цикъла в последния елемент на масива се записва \0 (това е причината да се използва „-1” за размер на буфера за четене) и се принтира този буфер. Сървърът може да изпрати повече от 127 бита, което ще означава, че буфера ще бъде презаписван и принтиран няколко пъти.

```
close(sock_fd);
```

Накрая се затваря сокета.

Задача 7: echoserver и echoclient

echoserver.c:

Използвани
библиотеки.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <signal.h>
#include <sys/socket.h>
#include <sys/wait.h>
#include <netinet/in.h>
```

```
// Signal handler to clean up finished child processes (reaps zombies)
void handle_sigchld() {
    // WNOHANG makes waitpid non-blocking so the main loop never stutters
    while (waitpid(-1, NULL, WNOHANG) > 0);
}
```

Създава се handler, който да трие приключилите дъщерни процеси. С използване на опцията WNOHANG основния цикъл никога не се блокира от handler-a.

```
void handle_client(int client_fd) {
    char buf[1024];
    ssize_t n;

    // Read blocks of 1024 bytes and echo them back verbatim
    while ((n = read(client_fd, buf, sizeof(buf))) > 0) {
        write(client_fd, buf, n);
    }

    close(client_fd);
}
```

Тази функция създава буфер, който съдържа данните изпратени от клиента и му ги връща.

```
if (argc != 2) {
    fprintf(stderr, "echoserver: missing port\n");
    return 1;
}
```

Във функцията main се проверява дали е подаден точно един параметър.

```
// Register the SIGCHLD handler
struct sigaction sa;
sa.sa_handler = handle_sigchld;
sigemptyset(&sa.sa_mask);
sa.sa_flags = SA_RESTART; // Prevents interrupted system calls (like
accept) from crashing
if (sigaction(SIGCHLD, &sa, NULL) < 0) {
    perror("echoserver: sigaction");
    return 1;
}
```

Създава се структура, която дефинира при какъв сигнал, кой handler да бъде извикан и какви настройки да бъдат използвани. SA_RESTART предотвратява прекъснати системни извиквания (като accept) да предизвика срив на програмата.

Сигналят SIGCHLD се изпраща от ядрото на Linux към бащин процес когато някой то дъщерните му процеси промени състоянието си, най-често когато дъщерния процес е терминиран или спре.

```
int server_fd = socket(AF_INET, SOCK_STREAM, 0);
if (server_fd < 0) {
    perror("echoserver");
    return 1;
}
```

Създава се сокет, от AF_INET семейството, което указва на сокета, че ще използва протоколния стек на Ipv4. Указваме, че искаме транспортният протокол да е TCP със SOCK_STREAM. Понеже единствения протокол в SOCK_STREAM е TCP, избираме първия (използвайки 0). Накрая проверяваме дали дали сокета е създаден успешно.


```
int opt = 1;
setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
```

Задават се параметри, които да бъдат използвани за сокета. Със `setsockopt` указваме, че искаме порта да бъде достъпен за използване, незабавно след затваряне на сокета.

```
struct sockaddr_in addr = {0};
addr.sin_family = AF_INET;
addr.sin_addr.s_addr = INADDR_ANY;
addr.sin_port = htons(atoi(argv[1]));
```

Създава се структура, която да зададе семейство, адреси, на които да слуша сокета, и порт.

```
if (bind(server_fd, (struct sockaddr *)&addr, sizeof(addr)) < 0) {
    perror("echoserver");
    close(server_fd);
    return 1;
}
```

Свързва се сокета с гореспоменатата структура.

```
if (listen(server_fd, 10) < 0) {  
    perror("echoserver");  
    close(server_fd);  
    return 1;  
}
```

Задава се сокета да слуша за клиенти и да позволява до десет клиента наведниж. Listen връща или 0 (сокета успешно е сложен в състояние на слушане), или -1 (когато има грешка).

```
while (1) {
```

Създава се безкраен цикъл.

```
int client_fd = accept(server_fd, NULL, NULL);  
if (client_fd < 0) {  
    // accept might fail if interrupted by our SIGCHLD signal handler  
    continue;  
}
```

Създава се клиентски сокет. Ако създаването се провали, заради сигнал handler-а на SIGCHLD, цикъла продължава със следващата итерация.

```

pid_t pid = fork();
if (pid == 0) {
    // --- CHILD PROCESS ---
    close(server_fd); // Child doesn't need the listener socket
    handle_client(client_fd);
    exit(0);          // Exit child cleanly when communication finishes
} else if (pid > 0) {
    // --- PARENT PROCESS ---
    close(client_fd); // Parent doesn't need this specific client descriptor
    anymore
} else {
    perror("echoserver: fork failed");
    close(client_fd);
}

```

Създава се дъщерен процес, който е копие на текущия. Блокът код под `pid==0` се изпълнява само на дъщерния процес. При него се затваря сървърния сокет и се извиква функцията `handle_client`, която прочита информацията изпратена от клиента и му я връща и затваря клиентския сокет. След това се връща изходен код 0.

Кодът под `pid > 0` се изпълнява само на бащиния процес. При него клиентския сокет се затваря.

Кодът под `else` се изпълнява когато има грешка при създаването.

```

close(server_fd);

```

При излизане от цикъла се затваря сървърния сокет.

echoclient.c:

Използвани
библиотеки.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <netdb.h>
```

```
if (argc != 4) {
    fprintf(stderr, "echoclient: missing arguments\n");
    return 1;
}
```

Проверява дали са подадени точно три аргумента.

```
char *hostname = argv[1];
int port = atoi(argv[2]);
char *message = argv[3];
```

Създадени са три променливи, които да запазят трите подадени параметъра. Порта се преобразува в int от string.

```

struct hostent *server = gethostbyname(hostname);
if (!server) {
    fprintf(stderr, "echoclient: host not found\n");
    return 1;
}

```

Прави DNS lookup, в случай, че е подадено име на хост, а не IP адрес. Ако е подаден IP адрес, той ще бъде запазен без DNS заявка.

```

int sock_fd = socket(AF_INET, SOCK_STREAM, 0);
if (sock_fd < 0) {
    perror("echoclient");
    return 1;
}

```

Създава се сокет, от AF_INET семейството, което указва на сокета, че ще използва протоколния стек на Ipv4. Указваме, че искаме транспортният протокол да е TCP със SOCK_STREAM. Понеже единствения протокол в SOCK_STREAM е TCP, избираме първия (използвайки 0). Накрая проверяваме дали дали сокета е създаден успешно.

```

struct sockaddr_in addr = {0};
addr.sin_family = AF_INET;
memcpy(&addr.sin_addr.s_addr, server->h_addr_list[0], server->h_length);
addr.sin_port = htons(port);

```

Създава се структура, която задава семейство, IP адреси, на които да слуша сокета, и порт.

```

if (connect(sock_fd, (struct sockaddr *)&addr, sizeof(addr)) < 0) {
    perror("echoclient");
    close(sock_fd);
    return 1;
}

```

Клиентския сокет се свързва със сървъра.

```
// Send the message string directly  
write(sock_fd, message, strlen(message));
```

Изпраща се подаденото съобщение към сървъра.

```
// Read back the echo response  
char buf[1024];  
ssize_t n = read(sock_fd, buf, sizeof(buf) - 1);  
if (n > 0) {  
    buf[n] = '\0';  
    printf("%s", buf);  
}
```

Прочита се отговора на сървъра и се принтира.

Затваря се сокета.

```
close(sock_fd);
```

Използвани източници:

<https://geeksforgeeks.com>

<https://cppreference.com>